Fundraising Platform — Complete Business and Domain Design

Complete business & domain design • fundraising operations • payments • reporting • APIs • UI • security • cloud • AI

Document Map

- Business & actors
- Three database responsibilities
- FundraisingDb domain
- PaymentDb domain
- ReportingDb / OLAP
- End-to-end business lifecycle
- Ownership & consistency
- Business states & capabilities
- API & Angular areas
- Roles, privacy & security placement
- Azure / DevOps placement
- AI, agents & MCP placement
- Reusable scenarios
- Document series & memory model

Purpose of This Document

This document explains the **business system before we write SQL Server code, .NET APIs, Azure infrastructure, Angular screens, or AI agents**.

The goal is to understand:

- what business we are building,
- who uses the system,
- what each part of the system does,
- why three databases exist,
- what every major business entity means,
- how those entities relate,
- how a donor moves through the system,
- how a gift becomes a payment,
- how payments are attempted, refunded, and settled,
- how fundraising employees interact with donors,
- how reporting receives data,
- what the major business workflows are,
- what future APIs and applications must support.

The project contains **three databases**:

EXAMPLE / FLOW

```
FundraisingDb
PaymentDb
ReportingDb
```

Their responsibilities are intentionally different:

EXAMPLE / FLOW

```
FundraisingDb
→ fundraising business operations

PaymentDb
→ money/payment processing

ReportingDb
→ analytics and reporting
```

That three-database boundary is already the foundation defined in the uploaded project design.

✦ The Business We Are Building

Imagine a fundraising organization such as:

EXAMPLE / FLOW

```
ABC Children's Foundation
```

The foundation raises money from:

EXAMPLE / FLOW

```
individual donors
companies
foundations
alumni
volunteers
supporters
```

Employees of the organization need software to manage the complete relationship with those people and organizations.

The platform needs to answer questions such as:

EXAMPLE / FLOW

```
Who is this donor?

Where does the donor live?

What has the donor given?

Which campaign motivated the donation?

Which fund should receive the money?

Has the donor actually paid?

Was the payment successful?

Was any money refunded?

Has the payment processor settled the money?

When did an employee last speak with this donor?

Does somebody need to follow up?

How much did we raise this month?

Which campaign is performing best?

Which funds receive the most giving?

What percentage of payments succeed?
```

```
What is our five-year giving trend?
```

The system therefore has three different responsibilities.

EXAMPLE / FLOW

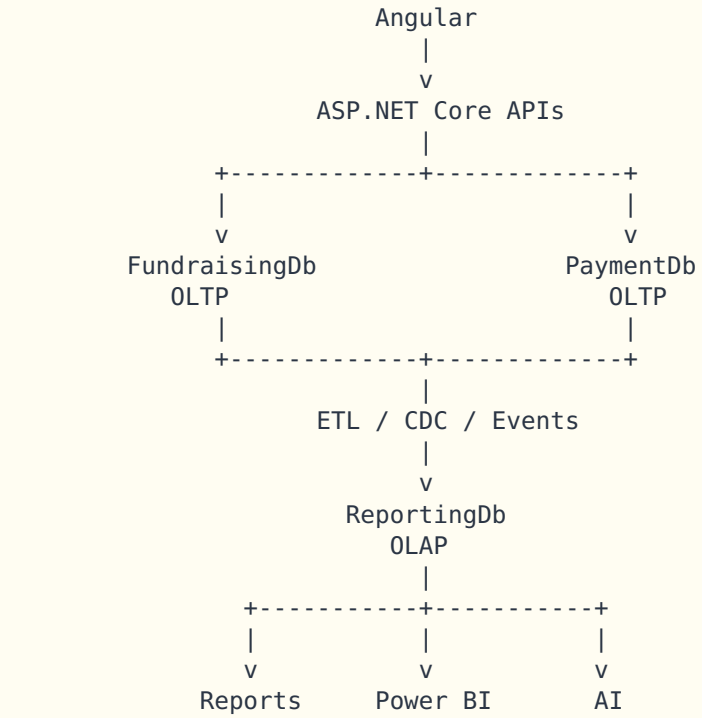
```
BUSINESS OPERATIONS
        |
        v
FundraisingDb

MONEY PROCESSING
        |
        v
PaymentDb

ANALYTICS
        |
        v
ReportingDb
```

The architecture in the project is intentionally:

EXAMPLE / FLOW



That separation between the two operational databases and the analytical database is part of the existing project design.

✦ The Most Important Business Principle

The three databases do **not** represent three unrelated applications.

They are three parts of one fundraising platform.

Think of them this way:

EXAMPLE / FLOW

```
FundraisingDb
=
What happened in fundraising?

PaymentDb
=
What happened to the money?

ReportingDb
=
What can we learn from everything that happened?
```

A donor may therefore participate in one business journey that crosses all three systems.

Example:

EXAMPLE / FLOW

```
John Smith decides to donate $1,000
                |
                v
        FundraisingDb
        records the Gift
                |
                v
        PaymentDb
        collects the $1,000
                |
                v
        ReportingDb
        analyzes the Gift
        and Payment
```

✦ The People and Systems That Use the Platform

Before discussing tables, understand the business actors.

Donor / Constituent

A donor is someone who may:

EXAMPLE / FLOW

```
make a donation
receive communications
attend events
have meetings with fundraisers
receive thank-you messages
support campaigns
```

But the project uses the more general word:

EXAMPLE / FLOW

```
Constituent
```

because a constituent is not necessarily already a donor.

A constituent could be:

EXAMPLE / FLOW

```
donor

prospect

alumnus

volunteer

board member

corporation

foundation
```

That broader meaning is defined in the project design.

✦ Fundraiser / Employee

An employee who uses the fundraising application is represented separately from a donor.

Examples:

EXAMPLE / FLOW

```
Fundraiser

Development Officer

Finance Employee

Fundraising Manager

Administrator
```

This distinction is important:

EXAMPLE / FLOW

```
AppUser
=
employee using the application

Constituent
=
person or organization
the employee works with
```

The uploaded design explicitly separates these concepts.

✦ Finance / Payment Operations Employee

A finance employee may need to:

EXAMPLE / FLOW

```
review successful payments

investigate failed payments

approve or review refunds

review settlements

reconcile processor payments

investigate discrepancies
```

This work belongs primarily around `PaymentDb`.

✦ Fundraising Manager

A fundraising manager needs information such as:

EXAMPLE / FLOW

```
Campaign progress

Top donors

Fund totals

Donor engagement

Open tasks

Fundraiser activity
```

Some information may come directly from `FundraisingDb`.

Long-term analytical reporting should come from:

EXAMPLE / FLOW

```
ReportingDb.
```

✦ System Administrator

An administrator manages things such as:

EXAMPLE / FLOW

```
organization configuration

staff access

system settings

monitoring

integration failures
```

Technical security implementation will be covered later in the .NET, SQL Server, and Azure documents.

✦ Payment Provider

The payment provider is an external service.

It may handle:

EXAMPLE / FLOW

```
card payment

digital wallet

direct debit

tokenization

authorization

payment confirmation

refund

settlement

fraud signals
```

The platform should normally avoid handling raw sensitive card credentials itself.

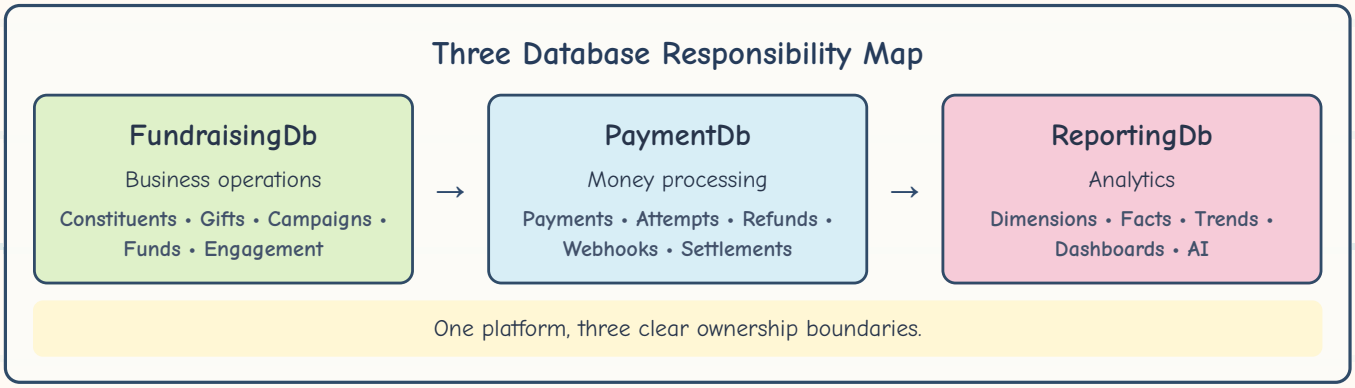
Instead:

EXAMPLE / FLOW

```
Donor
   |
   v
Hosted / Secure Payment Experience
   |
   v
Payment Provider
   |
   v
Token / Provider Reference
   |
   v
Our Payment API
   |
   v
PaymentDb
```

The project specifically states that PaymentDb should keep safe provider references rather than raw PAN, CVV, PIN, track data, or plaintext secrets.

✦ The Three Database Responsibilities



✦ FundraisingDb — Business Fundraising Truth

`FundraisingDb` is the primary fundraising operational database.

It answers:

EXAMPLE / FLOW

```
Who is the donor?

Where does the donor live?

Which campaign are we raising money for?

Where should donated money go?

What gift did the donor make?

Who interacted with the donor?

What follow-up work remains?
```

This is the project's main OLTP fundraising database.

It contains these 12 tables:

EXAMPLE / FLOW

```
core.Tenant
core.AppUser

constituent.Constituent
constituent.Address
constituent.ConstituentAddress

fundraising.Campaign
fundraising.Fund
fundraising.Gift
fundraising.GiftAllocation

engagement.Interaction
engagement.Task

integration.OutboxMessage
```

✦ PaymentDb — Payment Processing Truth

`PaymentDb` owns actual payment processing.

It deals with:

EXAMPLE / FLOW

```
merchant configuration

payment creation

payment method references

processing attempts

fraud decisions

refunds

provider webhooks

settlements

reconciliation

duplicate-request prevention
```

The project defines ten PaymentDb tables.

They are:

EXAMPLE / FLOW

```
payment.MerchantAccount
payment.Payment
payment.PaymentMethod
payment.PaymentAttempt
payment.FraudAssessment
payment.Refund
payment.WebhookEvent
payment.Settlement
payment.SettlementItem
payment.IdempotencyRequest
```

✦ ReportingDb — Analytical Truth

`ReportingDb` does not run normal fundraising or payment operations.

It exists for questions such as:

EXAMPLE / FLOW

```
How much did we raise this year?

Which campaigns perform best?

Which funds receive the most money?

How many payments succeed?

What is the refund amount?

What is donor retention?

What are monthly giving trends?
```

It is an analytical OLAP database rather than another transactional application database.

Its eight tables are:

EXAMPLE / FLOW

```
DimTenant
DimConstituent
DimCampaign
DimFund
DimDate

FactGift
FactPayment
FactInteraction
```

✦ FundraisingDb Business Model

Tenant

A `Tenant` represents one organization using the SaaS fundraising platform.

Examples:

EXAMPLE / FLOW

```
ABC Children's Foundation

XYZ University

Hope Hospital Foundation
```

This means your platform can support several independent organizations.

Example:

EXAMPLE / FLOW

```
Tenant 1
ABC Children's Foundation

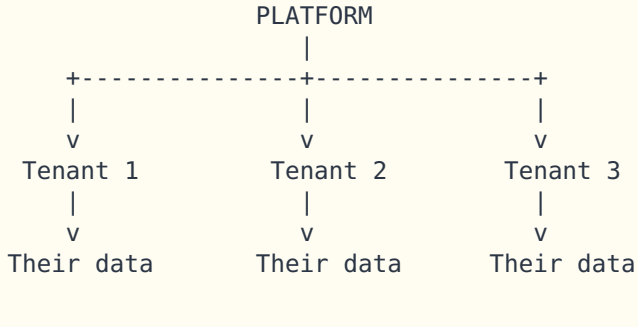
Tenant 2
XYZ University

Tenant 3
Hope Hospital Foundation
```

Each organization's business information must remain logically separated.

Conceptually:

EXAMPLE / FLOW



`Tenant` is therefore the root of the multi-tenant business model.

✦ AppUser

`AppUser` represents an employee of a tenant.

For example:

EXAMPLE / FLOW

```
Tenant:
ABC Children's Foundation

Employee:
Sarah Johnson

Role in business:
Senior Fundraiser
```

Sarah is not the donor.

Sarah is an employee who works with donors.

Conceptually:

EXAMPLE / FLOW

```
Tenant
   |
   | employs
   v
AppUser
```

Employees may:

EXAMPLE / FLOW

```
create campaigns

create funds

record gifts

call donors

create interactions

create tasks

assign work
```

The project intentionally keeps authentication credentials outside this business table; identity is expected to come from Microsoft Entra ID rather than storing passwords or tokens in `AppUser`.

✦ Constituent

A `Constituent` represents a person or organization related to fundraising.

Example:

EXAMPLE / FLOW

```
Constituent
John Smith

Type
Person
```

Another:

EXAMPLE / FLOW

```
Constituent
Microsoft Corporation

Type
Organization
```

A constituent can exist before becoming a donor.

Example:

EXAMPLE / FLOW

```
Prospect created
      |
      v
Fundraiser contacts prospect
      |
      v
Prospect becomes donor
      |
      v
Gift recorded
```

This is important because fundraising is relationship management, not merely payment processing.

✦ Address

An `Address` stores the physical address.

Example:

EXAMPLE / FLOW

```
123 Main Street
Suite 200
Richmond
Virginia
23220
US
```

It is separated from the constituent because the relationship between people and addresses has its own business meaning.

✦ ConstituentAddress

`ConstituentAddress` connects:

EXAMPLE / FLOW

```
Constituent
```

with:

EXAMPLE / FLOW

```
Address.
```

It also tells us how the address is used.

Example:

EXAMPLE / FLOW

```
John Smith
     |
     v
123 Main Street
     |
     v
Address Type = Home
Primary = Yes
```

A constituent may have:

EXAMPLE / FLOW

```
home address

work address

mailing address
```

and addresses may change over time.

The project therefore models the relationship explicitly rather than simply placing every address field directly on the Constituent record.

✦ Campaign

A `Campaign` answers:

> **Why are we raising money?**

Examples:

EXAMPLE / FLOW

```
Annual Giving 2026

Cancer Research Campaign

Hospital Expansion Campaign

Giving Tuesday
```

A campaign represents the fundraising effort or initiative.

Example:

EXAMPLE / FLOW

```
Campaign:
Cancer Research Campaign

Goal:
$500,000

Start:
January 1

End:
December 31
```

Many gifts may be associated with the same campaign.

EXAMPLE / FLOW

```
Campaign
    |
    +---- Gift 1
    |
    +---- Gift 2
    |
    +---- Gift 3
    |
    +---- Gift 4
```

The project describes `Campaign` precisely as the reason fundraising is happening.

✦ Fund

A `Fund` answers a different question:

> **Where is the donated money allowed or intended to go?**

Examples:

EXAMPLE / FLOW

```
General Fund

Cancer Research Fund

Scholarship Fund

Patient Assistance Fund
```

Do not confuse:

EXAMPLE / FLOW

```
Campaign
```

with:

EXAMPLE / FLOW

```
Fund.
```

The easiest business distinction is:

EXAMPLE / FLOW

```
Campaign
=
WHY are we raising the money?

Fund
=
WHERE can the money be used?
```

That distinction is explicitly made in the existing design.

✦ Gift

`Gift` is the primary fundraising transaction.

Example:

EXAMPLE / FLOW

```
John Smith donated $1,000
to the 2026 Annual Campaign.
```

A Gift answers:

EXAMPLE / FLOW

```
Who donated?

How much?

When?

Which campaign?

What type of gift?
```

Conceptually:

EXAMPLE / FLOW

```
John Smith
     |
     | gives
     v
Gift
$1,000
     |
     | associated with
     v
2026 Annual Campaign
```

A Gift may have a campaign.

The project's Gift relationship connects the constituent, campaign, employee who recorded it, and later its allocations.

✦ GiftAllocation

`GiftAllocation` answers:

> **How should the Gift amount be divided between Funds?**

Suppose:

EXAMPLE / FLOW

```
Gift
$1,000
```

The donor says:

EXAMPLE / FLOW

```
$700
→ Cancer Research Fund

$300
→ Patient Assistance Fund
```

The system stores:

EXAMPLE / FLOW

```
Gift 10000
     |
     +---- Allocation
     |     $700
     |       |
     |       v
     |     Cancer Research Fund
     |
     +---- Allocation
           $300
             |
             v
           Patient Assistance Fund
```

That gives the important relationship:

EXAMPLE / FLOW

```
Gift
 M
 |
 v
GiftAllocation
 ^
 |
 M
Fund
```

So conceptually:

EXAMPLE / FLOW

```
Gift M:N Fund
```

through:

EXAMPLE / FLOW

```
GiftAllocation.
```

The project defines the expected business rule that allocation totals should equal the Gift total.

Example:

EXAMPLE / FLOW

```
Gift
$1,000

Allocations
$700
+
$300
=
$1,000
```

✦ Interaction

An `Interaction` records communication or engagement between an employee and constituent.

Examples:

EXAMPLE / FLOW

```
phone call

email

meeting

donor visit

event conversation
```

Suppose Sarah calls John:

EXAMPLE / FLOW

```
Constituent:
John Smith

Fundraiser:
Sarah Johnson

Type:
Phone Call

Subject:
Discussed Annual Giving campaign
```

Relationship:

EXAMPLE / FLOW

```
Constituent
      |
      v
Interaction
      ^
      |
AppUser
```

The interaction therefore answers:

EXAMPLE / FLOW

```
Who did we communicate with?

Which employee communicated?

When?

How?

What was discussed?
```

✦ Task

A `Task` represents work that should happen.

Examples:

EXAMPLE / FLOW

```
Call John Smith

Send thank-you letter

Schedule meeting

Review donor history
```

A Task can have:

EXAMPLE / FLOW

```
Constituent

CreatedByUser

AssignedToUser

Due date

Priority

Completion date
```

For example:

EXAMPLE / FLOW

```
Sarah creates task
        |
        v
"Send thank-you letter"
        |
        v
Assigned to David
        |
        v
Related to John Smith
```

The design deliberately separates:

EXAMPLE / FLOW

```
CreatedByUserId
```

from:

EXAMPLE / FLOW

```
AssignedToUserId.
```

One employee may create the work while another performs it.

✦ OutboxMessage

`OutboxMessage` is an internal integration record.

It represents:

> Something important happened in FundraisingDb and another system needs to know about it.

For example:

EXAMPLE / FLOW

```
Gift created
      |
      v
OutboxMessage created
      |
      v
Fundraising transaction commits
      |
      v
Background worker
      |
      v
Azure Service Bus
      |
      v
Other service
```

Example event:

EXAMPLE / FLOW

```
GiftCreated
```

The project explicitly defines this table for reliable event publishing and later Azure Service Bus/ microservices use.

This table will become extremely important in the later:

EXAMPLE / FLOW

```
.NET document

Azure document

Microservices document

AI/event automation work
```

but at the business level its purpose is simply:

> **Do not lose important business events after a successful transaction.**

✦ FundraisingDb Complete Relationship

The business relationships are:

EXAMPLE / FLOW

```
Tenant
|
```

```
├── AppUser
│
├── Constituent
│   │
│   ├── ConstituentAddress
│   │       │
│   │       └── Address
│   │
│   ├── Gift
│   │    │
│   │    ├── Campaign
│   │    │
│   │    └── GiftAllocation
│   │                │
│   │                └── Fund
│   │
│   ├── Interaction
│   │       │
│   │       └── AppUser
│   │
│   └── Task
│           │
│           └── AppUser
│
├── Campaign
│
└── Fund
```

The central fundraising chain is:

EXAMPLE / FLOW

```
Constituent
    |
    v
Gift
    |
    v
GiftAllocation
    |
    v
Fund
```

with:

EXAMPLE / FLOW

```
Gift
 |
 v
Campaign
```

This is also the project's documented core relationship.

The easiest memory model is:

EXAMPLE / FLOW

```
WHO?
→ Constituent

HOW MUCH?
→ Gift

WHY?
→ Campaign

WHERE?
→ Fund

HOW DIVIDED?
→ GiftAllocation
```

✦ PaymentDb Business Model

FundraisingDb records the business fact:

EXAMPLE / FLOW

```
John Smith gave $1,000.
```

That does not automatically mean:

EXAMPLE / FLOW

```
$1,000 successfully reached
the organization's bank account.
```

That second problem belongs to:

EXAMPLE / FLOW

```
PaymentDb.
```

✦ MerchantAccount

A `MerchantAccount` represents the organization's configuration with a payment processor.

Example:

EXAMPLE / FLOW

```
Tenant:
ABC Children's Foundation

Provider:
ExamplePay

Default currency:
USD

Card enabled:
Yes

Digital wallet:
Yes
```

It tells PaymentDb:

> Which external payment-processing configuration should be used for this organization?

Sensitive provider passwords and secrets should not be stored here; the project assigns those to Azure Key Vault.

✦ Payment

`Payment` represents:

> **Collect this amount of money for this fundraising Gift.**

Example:

EXAMPLE / FLOW

```
FundraisingDb

GiftId:
10000

Gift:
$1,000
```

PaymentDb creates:

EXAMPLE / FLOW

```
PaymentId:
90000

FundraisingGiftId:
10000

Amount:
$1,000

Status:
Pending
```

The important concept is:

EXAMPLE / FLOW

```
Gift
≠
Payment.
```

A Gift is the fundraising commitment/business transaction.

A Payment is the financial collection process.

This matters because:

EXAMPLE / FLOW

```
Gift may exist
while Payment fails.
```

Example:

EXAMPLE / FLOW

```
John commits to $1,000
        |
        v
Gift exists
        |
        v
Card declined
        |
        v
Payment failed
```

Fundraising history and payment-processing history are different things.

✦ Why FundraisingGiftId Is Not a Database Foreign Key

`PaymentDb.Payment` can contain:

EXAMPLE / FLOW

```
FundraisingGiftId
```

which logically refers to:

EXAMPLE / FLOW

```
FundraisingDb.fundraising.Gift.GiftId.
```

But the architecture deliberately avoids cross-database/service foreign keys.

Instead:

EXAMPLE / FLOW

```
FundraisingDb
owns Gift.

PaymentDb
owns Payment.

Applications/events
connect their identities.
```

That boundary is important for future microservices.

✦ PaymentMethod

`PaymentMethod` describes the safe payment method reference used during payment.

Examples:

EXAMPLE / FLOW

```
Card

Direct Debit

Digital Wallet
```

But the platform should not store raw card secrets.

It may store things like:

EXAMPLE / FLOW

```
Provider token

Provider payment method ID

Card brand

Last 4 digits

Expiration month/year

Wallet type
```

For example:

EXAMPLE / FLOW

```
Payment Method
Visa ****4242

Provider Token
tok_abc123
```

The payment provider retains the sensitive credential.

✦ PaymentAttempt

A Payment may require more than one processing attempt.

Example:

EXAMPLE / FLOW

```
Payment
$1,000
   |
   +---- Attempt 1
   |     Declined
   |
   +---- Attempt 2
   |     Timeout
   |
   +---- Attempt 3
         Success
```

That is why Payment and PaymentAttempt should not be the same record.

Payment describes the overall business collection.

PaymentAttempt describes each technical/business processing attempt.

Possible attempt outcomes might include:

EXAMPLE / FLOW

```
started

authorized

declined

failed

timed out

succeeded
```

The exact status model will be finalized later in the SQL/.NET design rather than creating unnecessary lookup tables now.

✦ FraudAssessment

Payment processing may include a risk/fraud decision.

Example:

EXAMPLE / FLOW

```
PaymentAttempt
      |
      v
FraudAssessment
```

Possible signals may include:

EXAMPLE / FLOW

```
transaction velocity

IP risk

country mismatch

unusual amount

multiple previous failures

provider fraud score
```

A decision might be:

EXAMPLE / FLOW

```
Allow

Review

Reject
```

The existing project supports multiple assessments if a provider re-evaluates a transaction.

✦ Refund

A `Refund` records money returned from a successful Payment.

Suppose:

EXAMPLE / FLOW

```
Payment
$1,000
```

Later:

EXAMPLE / FLOW

```
Refund 1
$200

Refund 2
$100
```

Payment status may eventually become:

EXAMPLE / FLOW

```
Partially Refunded
```

Total successful refunds should not exceed the Payment amount.

The project explicitly includes this business rule.

Conceptually:

EXAMPLE / FLOW

```
Payment $1,000
       |
       +---- Refund $200
       |
       +---- Refund $100
```

Total:

EXAMPLE / FLOW

```
$300 refunded
$700 net remaining
```

✦ WebhookEvent

Payment providers often communicate asynchronously.

For example:

EXAMPLE / FLOW

```
Our API requests payment
        |
        v
Provider processes it
        |
        v
Provider sends webhook
        |
        v
Our Webhook endpoint
        |
        v
WebhookEvent
        |
        v
Payment state updated
```

Examples of provider events:

EXAMPLE / FLOW

```
payment.succeeded

payment.failed

refund.succeeded

settlement.created
```

The system should record the external provider event ID.

A critical business requirement is:

> **The same webhook must not be processed twice.**

That is why the project requires uniqueness around provider and provider event ID.

✦ Settlement

A successful Payment does not necessarily mean the money has already been transferred into the nonprofit's bank account.

A processor may collect many transactions and later send a settlement.

Example:

EXAMPLE / FLOW

```
Payments collected:
$10,000

Processor fees:
$300

Net settlement:
$9,700
```

`Settlement` therefore represents money that the processor sends to the organization.

The project explicitly models:

EXAMPLE / FLOW

```
GrossAmount

FeeAmount

NetAmount
```

with:

EXAMPLE / FLOW

```
NetAmount =
GrossAmount - FeeAmount
```

✦ SettlementItem

`SettlementItem` connects individual Payments to a Settlement.

Example:

EXAMPLE / FLOW

```
Settlement
$9,700
    |
    +---- Payment A
    |
    +---- Payment B
    |
    +---- Payment C
    |
    +---- Payment D
```

This lets finance answer:

EXAMPLE / FLOW

```
Which individual payments
were included in this bank settlement?

What fee belonged to each payment?

Are there missing transactions?

Does processor reporting reconcile
with our records?
```

✦ IdempotencyRequest

`IdempotencyRequest` prevents accidental duplicate payment processing.

This is extremely important.

Suppose Angular sends:

EXAMPLE / FLOW

```
Create payment for Gift 10000
```

Payment succeeds.

But the HTTP response is lost.

Angular retries.

Without idempotency:

EXAMPLE / FLOW

```
Request 1
→ charge $1,000

Request 2
→ charge another $1,000
```

That would be a serious business error.

Instead:

EXAMPLE / FLOW

```
Request 1

IdempotencyKey:
ABC123

        |
        v

Payment created
```

Then retry:

EXAMPLE / FLOW

```
Request 2

IdempotencyKey:
ABC123

        |
        v

Existing result returned

NO second charge
```

The project already requires uniqueness for the tenant/idempotency key combination.

✦ PaymentDb Complete Relationship

EXAMPLE / FLOW

```
MerchantAccount
       |
       | 1:M
       v
    Payment
```

```
       |
       +----------------------+
       |                      |
       v                      v
PaymentAttempt              Refund
       |
       +---- PaymentMethod
       |
       v
FraudAssessment


Payment
   |
   v
SettlementItem
   |
   v
Settlement


IdempotencyRequest
       |
       v
    Payment


External Payment Provider
       |
       v
WebhookEvent
       |
       v
Payment lifecycle update
```

That relationship structure comes directly from the existing PaymentDb design.

✦ ReportingDb Business Model

FundraisingDb and PaymentDb are operational systems.

They are designed primarily to:

EXAMPLE / FLOW

```
insert

update

process transactions

handle concurrent users

respond quickly to application requests
```

ReportingDb serves a different type of work.

It is designed to answer questions over:

EXAMPLE / FLOW

```
months

years

large numbers of gifts

large numbers of payments

historical donor information
```

The project specifically distinguishes FundraisingDb and PaymentDb as OLTP and ReportingDb as OLAP.

✦ Why ReportingDb Is Separate

Suppose management wants:

EXAMPLE / FLOW

```
Five-year giving trends
by tenant
by campaign
by state
by month
by fund.
```

Running huge analytical queries directly against the same database accepting live donations may create unnecessary competition with transactional work.

Instead:

EXAMPLE / FLOW

```
FundraisingDb
      |
      +----+
           |
           v
       ReportingDb

PaymentDb
      |
      +----+
```

ReportingDb receives analytical copies/transformations of information.

✦ Dimension vs Fact

The ReportingDb uses a star-schema style.

To understand it simply:

EXAMPLE / FLOW

```
Dimension
=
Who / What / Where / When

Fact
=
What measurable business event happened?
```

Examples:

EXAMPLE / FLOW

```
DimConstituent
→ WHO donated?

DimCampaign
→ WHICH campaign?

DimFund
→ WHICH fund?

DimDate
→ WHEN?

FactGift
→ HOW MUCH was donated?
```

✦ DimTenant

`DimTenant` represents a tenant in the reporting warehouse.

Unlike the transactional Tenant table, the dimension can keep historical versions.

For example:

EXAMPLE / FLOW

```
Tenant name originally:
ABC Children's Foundation

Later:
ABC Children's Health Foundation
```

Reporting may need to understand how the business dimension looked historically.

The project includes:

EXAMPLE / FLOW

```
EffectiveFrom

EffectiveTo

IsCurrent
```

for this type of historical handling.

✦ DimConstituent

`DimConstituent` provides reporting-friendly donor information.

Unlike the normalized FundraisingDb model:

EXAMPLE / FLOW

```
Constituent
     |
     v
ConstituentAddress
     |
     v
Address
```

ReportingDb can intentionally flatten information:

EXAMPLE / FLOW

```
DimConstituent
----------------
DisplayName
City
StateProvince
CountryCode
```

That denormalization is intentional in the project.

Reporting is optimized for questions, not transactional editing.

✦ DimCampaign

Provides reporting information about campaigns.

Examples:

EXAMPLE / FLOW

```
Campaign name

Goal amount

Start date

End date
```

It can retain historical versions if campaign characteristics change.

✦ DimFund

Provides reporting information about Funds.

Examples:

EXAMPLE / FLOW

```
Fund code

Fund name

Restricted / unrestricted
```

✦ DimDate

`DimDate` represents calendar information.

Example:

EXAMPLE / FLOW

```
Date:
2026-09-22

Day:
22

Month:
September

Quarter:
3

Year:
2026

Weekend:
No
```

This makes analytical questions easier.

Instead of calculating calendar logic in every report, reporting queries can use the Date dimension.

The project specifically includes a reusable date dimension for this purpose.

✦ FactGift

`FactGift` stores measurable fundraising facts.

The critical design is its **grain**.

In this project, a Gift split between Funds may appear as multiple FactGift records at the allocation level.

Example:

EXAMPLE / FLOW

```
Gift
$1,000

Allocation
Fund A
$700

Allocation
Fund B
$300
```

Reporting facts:

EXAMPLE / FLOW

```
Gift 10000
Fund A
GiftAmount = 700
```

and:

EXAMPLE / FLOW

```
Gift 10000
Fund B
GiftAmount = 300
```

Why?

Because management needs questions such as:

EXAMPLE / FLOW

```
How much money did Fund A receive?

How much did Fund B receive?
```

The project explicitly defines this allocation-grain approach.

✦ FactPayment

`FactPayment` is optimized for payment analytics.

It can answer:

EXAMPLE / FLOW

```
How much money was processed?

How much was refunded?

How much did processors charge in fees?

What was the net amount?

Which payment methods succeed most often?

How many attempts do payments require?

What percentage fail?
```

The existing design includes measures for gross amount, refund amount, fee amount, net amount, attempts, successes, and failures.

✦ FactInteraction

`FactInteraction` allows analytical questions around fundraising engagement.

Examples:

EXAMPLE / FLOW

```
How many donor calls occurred this month?

How many meetings did each fundraiser conduct?

Do highly engaged constituents give more?

How much engagement happens before large gifts?
```

The project explicitly includes these kinds of analytical questions.

✦ Reporting Star Relationship

Fundraising:

EXAMPLE / FLOW

```
                    DimConstituent
                          |
                          |
DimCampaign -------- FactGift -------- DimDate
                          |
                          |
                       DimFund
                          |
                       DimTenant
```

Payment:

EXAMPLE / FLOW

```
DimTenant
    |
FactPayment
    |
 DimDate
```

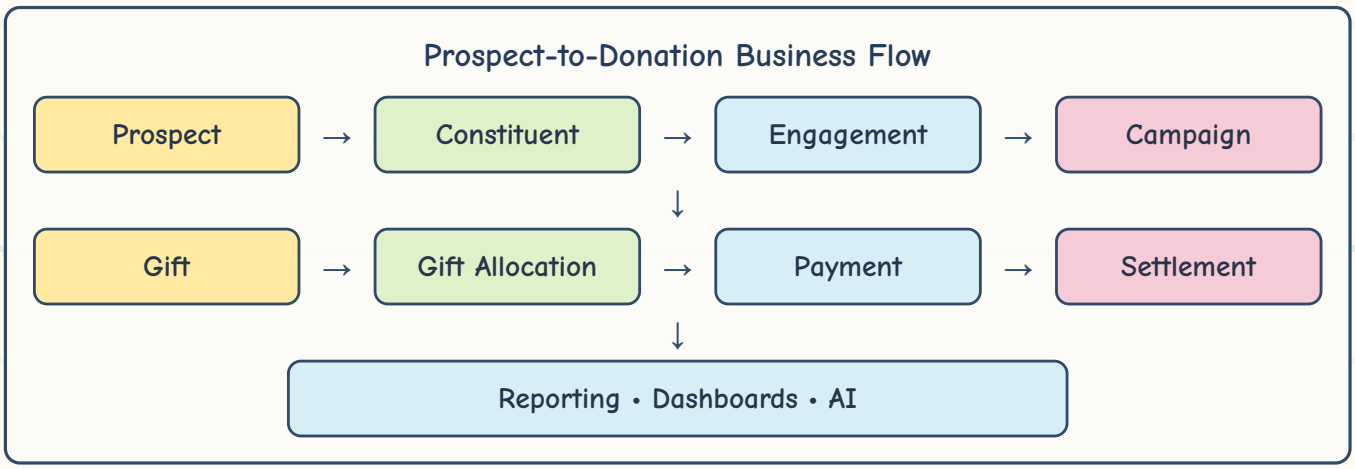
Interactions:

EXAMPLE / FLOW

```
DimTenant
    |
FactInteraction
    |
    +---- DimConstituent
    |
    +---- DimDate
```

This basic star-schema structure is defined in the project design.

✦ Complete Business Lifecycle — From Prospect to Donation



Now connect the entities into a real business process.

Imagine:

EXAMPLE / FLOW

```
John Smith
```

has never donated before.

A fundraiser meets him at an event.

Constituent Creation

Sarah creates:

EXAMPLE / FLOW

```
Constituent

John Smith
```

and records:

EXAMPLE / FLOW

```
Email

Phone

Address
```

Business flow:

EXAMPLE / FLOW

```
New prospect
      |
      v
Constituent
      |
      v
Address linked
```

At this point:

EXAMPLE / FLOW

```
Gift = none

Payment = none
```

John is simply part of the organization's relationship network.

✦ Engagement Before Donation

Sarah calls John.

The system records:

EXAMPLE / FLOW

```
Interaction

Type:
Phone Call

Fundraiser:
Sarah

Subject:
Cancer Research Campaign
```

Sarah creates:

EXAMPLE / FLOW

```
Task

Follow up with John next Friday
```

Flow:

EXAMPLE / FLOW

```
Constituent
      |
      +---- Interaction
      |
      +---- Task
```

This is typical CRM/fundraising work.

✦ Campaign Presentation

Sarah discusses:

EXAMPLE / FLOW

```
2026 Cancer Research Campaign
```

Business meaning:

EXAMPLE / FLOW

```
WHY are we asking John for money?
```

Answer:

EXAMPLE / FLOW

```
Cancer Research Campaign
```

✦ Gift Creation

John decides to donate:

EXAMPLE / FLOW

```
$1,000
```

The organization records:

EXAMPLE / FLOW

```
Gift

Constituent:
John Smith

Campaign:
2026 Cancer Research Campaign

Amount:
$1,000
```

✦ Gift Allocation

John wants:

EXAMPLE / FLOW

```
$700
→ Cancer Research Fund

$300
→ Patient Assistance Fund
```

The business record becomes:

EXAMPLE / FLOW

```
John Smith
    |
    v
Gift $1,000
    |
    +---- Cancer Research Campaign
    |
    +---- GiftAllocation $700
    |         |
    |         v
    |    Cancer Research Fund
    |
    +---- GiftAllocation $300
              |
              v
       Patient Assistance Fund
```

✦ Event Creation

During the same reliable fundraising operation:

EXAMPLE / FLOW

```
Gift
+
GiftAllocations
+
OutboxMessage
```

are recorded.

The Outbox message might represent:

EXAMPLE / FLOW

```
GiftCreated
```

The existing project explicitly uses this transaction/outbox pattern.

✦ Payment Begins

A background process or application workflow reacts to:

EXAMPLE / FLOW

```
GiftCreated.
```

PaymentDb creates:

EXAMPLE / FLOW

```
Payment

FundraisingGiftId:
10000

Amount:
$1,000

Status:
Pending
```

The existing project describes this fundraising-to-payment flow.

✦ Secure Payment Collection

John enters payment information through a secure payment provider experience.

Example:

EXAMPLE / FLOW

```
Visa ****4242
```

The payment provider returns something such as:

EXAMPLE / FLOW

```
ProviderToken
```

PaymentDb stores the token/reference, not raw sensitive card data.

✦ Payment Attempt

PaymentDb records:

EXAMPLE / FLOW

```
PaymentAttempt #1
```

Possible flow:

EXAMPLE / FLOW

```
Payment
     |
     v
PaymentAttempt
     |
     v
FraudAssessment
     |
     v
Provider authorization
```

If successful:

EXAMPLE / FLOW

```
Payment.Status
=
Succeeded
```

If not:

EXAMPLE / FLOW

```
Payment.Status
=
Failed
```

or it may allow another attempt.

✦ Provider Webhook

Even after our API request finishes, the provider might send asynchronous confirmation.

Example:

EXAMPLE / FLOW

```
Payment succeeded
       |
       v
Provider sends webhook
       |
       v
WebhookEvent stored
       |
       v
Signature verified
       |
       v
Payment updated
```

Duplicate provider events must not result in duplicate business processing.

✦ Settlement

Later:

EXAMPLE / FLOW

```
Payment provider
       |
       v
Settlement batch
       |
       v
Organization bank account
```

PaymentDb records:

EXAMPLE / FLOW

```
Settlement

SettlementItems
```

Now finance can reconcile:

EXAMPLE / FLOW

```
what we expected

versus

what the payment processor transferred.
```

✦ Refund Scenario

Suppose John accidentally donated too much.

Payment:

EXAMPLE / FLOW

```
$1,000
```

Refund:

EXAMPLE / FLOW

```
$200
```

Flow:

EXAMPLE / FLOW

```
Payment
$1,000
   |
   v
Refund Request
$200
   |
   v
Provider
   |
   v
Refund completed
   |
   v
Payment becomes
Partially Refunded
```

ReportingDb eventually receives updated payment analytics.

✦ Thank-You Workflow

A successful Gift may trigger a follow-up Task:

EXAMPLE / FLOW

```
Send thank-you letter
```

and later an Interaction:

EXAMPLE / FLOW

```
Thank-you call completed.
```

This gives the platform a complete relationship lifecycle, rather than stopping at payment.

✦ Reporting Flow

FundraisingDb now contains:

EXAMPLE / FLOW

```
John

Campaign

Gift

GiftAllocation

Interaction
```

PaymentDb contains:

EXAMPLE / FLOW

```
Payment

PaymentAttempt

Settlement

possibly Refund
```

Analytical processing moves appropriate information into ReportingDb:

EXAMPLE / FLOW

```
FundraisingDb --------+
                      |
                      +---- ETL / CDC / Events
                      |
PaymentDb ------------+
                      |
                      v
                 ReportingDb
```

The project already defines this data movement pattern and possible future technologies such as CDC, Azure Data Factory, Service Bus events, Azure Functions, workers, and ETL pipelines.

✦ Management Dashboard

Management might now see:

EXAMPLE / FLOW

```
September Giving
$2,850,000

Payment Success
96.8%

Top Campaign
Cancer Research

Top Fund
Cancer Research Fund

Refunds
$27,500

Active Donors
18,420
```

Those values should primarily come from:

EXAMPLE / FLOW

```
ReportingDb
```

for substantial analytical workloads.

✦ Business Ownership Rules

The project needs clear ownership.

FundraisingDb owns

EXAMPLE / FLOW

```
Tenant

AppUser

Constituent

Address

Campaign

Fund

Gift

GiftAllocation

Interaction

Task
```

If a donor's name changes:

EXAMPLE / FLOW

```
FundraisingDb
```

is the authoritative business system.

✦ PaymentDb owns

EXAMPLE / FLOW

```
Payment

PaymentMethod references

PaymentAttempt

FraudAssessment

Refund

WebhookEvent

Settlement

Idempotency
```

If payment status changes:

EXAMPLE / FLOW

```
PaymentDb
```

is authoritative.

✦ ReportingDb owns

EXAMPLE / FLOW

```
analytical dimensions

analytical facts

historical reporting representation
```

But it should not become the operational source of truth.

The uploaded architecture explicitly states that each database owns its own data and that ReportingDb is read-focused.

✦ Why We Do Not Put Everything in One Database

Technically, one SQL Server database could contain all these tables.

But the separation teaches real architecture.

It allows:

EXAMPLE / FLOW

```
Fundraising
```

and:

EXAMPLE / FLOW

```
Payment
```

to evolve independently.

It also keeps:

EXAMPLE / FLOW

```
analytics
```

away from normal transactional writes.

Later, this prepares you for:

EXAMPLE / FLOW

```
microservices

events

eventual consistency

independent deployment

independent scaling

separate security boundaries
```

✦ No Cross-Database Foreign Keys

One of the project's most important architecture rules is:

> Do not create cross-database foreign keys between separate service/database boundaries.

For example:

EXAMPLE / FLOW

```
FundraisingDb.Gift.GiftId
            |
            | logical ID/reference
            v
PaymentDb.Payment.FundraisingGiftId
```

But not:

EXAMPLE / FLOW

```
PaymentDb foreign key
directly referencing
FundraisingDb.Gift.
```

The relationship is maintained through:

EXAMPLE / FLOW

```
IDs

events

application workflows

service communication
```

rather than hard database coupling.

✦ No Giant Transaction Across All Three Databases

Another major rule:

Do not create:

EXAMPLE / FLOW

```
BEGIN TRANSACTION

Insert FundraisingDb.Gift

Insert PaymentDb.Payment

Insert ReportingDb.FactGift

COMMIT
```

as one giant distributed business operation.

Instead:

EXAMPLE / FLOW

```
FundraisingDb transaction
      |
      v
Outbox Event
      |
      v
Payment workflow
      |
      v
Reporting integration
```

Each service/database controls its own transaction.

The project explicitly recommends transactions remain inside a database/service boundary and use events/eventual consistency between services.

✦ Business Consistency vs Eventual Consistency

Inside one FundraisingDb transaction:

EXAMPLE / FLOW

```
Gift
+
GiftAllocation
+
OutboxMessage
```

should be immediately consistent.

But:

EXAMPLE / FLOW

```
FundraisingDb Gift
```

and:

EXAMPLE / FLOW

```
PaymentDb Payment
```

may temporarily be in different stages.

Example:

EXAMPLE / FLOW

```
10:00:00
Gift created

10:00:01
GiftCreated event published

10:00:02
Payment created
```

That short delay is acceptable if the workflow is designed correctly.

This is:

EXAMPLE / FLOW

```
eventual consistency.
```

You will learn the technical implementation later.

✦ Major Business States

The project should think in terms of business lifecycles.

Exact numeric values will be designed in SQL/.NET later.

Constituent lifecycle

Conceptually:

EXAMPLE / FLOW

```
Prospect
    |
    v
Active Relationship
    |
    +---- Donor
    |
    +---- Volunteer
    |
    +---- Other Engagement
    |
    v
Inactive / Archived
```

The existing source keeps Constituent status intentionally simple rather than creating a status table for every state.

✦ Campaign lifecycle

Recommended project lifecycle:

EXAMPLE / FLOW

```
Draft
   |
   v
Scheduled
   |
   v
Active
   |
   v
Completed
   |
   v
Archived
```

This is a recommended application-level extension; the uploaded database design currently gives Campaign dates and business fields but does not define this complete lifecycle.

✦ Gift lifecycle

The source model separates the Gift business record from actual Payment status.

Therefore a useful conceptual lifecycle is:

EXAMPLE / FLOW

```
Gift recorded
     |
     v
Payment requested
     |
     +---- Payment succeeded
     |
     +---- Payment failed
     |
     +---- Payment partially refunded
     |
     +---- Payment refunded
```

Do not overload Gift itself with all payment-provider state.

✦ Payment lifecycle

The source proposes statuses such as:

EXAMPLE / FLOW

```
Pending

Processing

Succeeded

Failed

Partially Refunded

Refunded

Cancelled
```

Conceptually:

EXAMPLE / FLOW

```
Pending
   |
   v
Processing
   |
   +----> Succeeded
   |
   +----> Failed
   |
   +----> Cancelled
```

After success:

EXAMPLE / FLOW

```
Succeeded
   |
   +----> Partially Refunded
   |
   +----> Refunded
```

✦ Task lifecycle

A Task may conceptually be:

EXAMPLE / FLOW

```
Open
   |
   v
In Progress
   |
   v
Completed
```

Your current data model can initially infer completion from:

EXAMPLE / FLOW

```
CompletedAtUtc IS NULL
```

versus:

EXAMPLE / FLOW

```
CompletedAtUtc IS NOT NULL.
```

Do not create extra tables unless the business eventually needs a more complicated workflow.

✦ Payment Attempt Lifecycle

Conceptually:

EXAMPLE / FLOW

```
Started
    |
    v
Provider Processing
    |
    +---- Authorized / Successful
    |
    +---- Declined
    |
    +---- Technical Failure
    |
    +---- Timeout
```

The exact status values belong in the SQL/.NET design.

✦ Refund Lifecycle

Conceptually:

EXAMPLE / FLOW

```
Requested
    |
    v
Processing
    |
    +---- Completed
    |
    +---- Failed
```

✦ Settlement Lifecycle

Conceptually:

EXAMPLE / FLOW

```
Expected
    |
    v
Received from provider
    |
    v
Reconciled
```

Again, exact implementation will be covered later.

✦ Complete Business Capabilities

Now think of the platform in business capabilities rather than database tables.

Tenant Administration

The platform should support:

EXAMPLE / FLOW

```
Create organization

Manage organization information

Activate/deactivate organization

Manage staff
```

✦ Constituent Management

Users need to:

EXAMPLE / FLOW

```
Create constituent

Edit constituent

Search constituents

View profile

Manage contact information

Manage addresses

View donor history

View interactions

View tasks

View giving history

View complete timeline
```

✦ Campaign Management

Users need to:

EXAMPLE / FLOW

Create campaign

Edit campaign

Set goal

Set dates

View campaign Gifts

View campaign performance

View historical campaign information

Compare campaigns

✦ Fund Management

Users need to:

EXAMPLE / FLOW

Create Fund

Mark restricted/unrestricted

Set Fund dates

View allocated giving

View Fund totals

✦ Gift Management

Users need to:

EXAMPLE / FLOW

Record Gift

Assign donor

Associate Campaign

Allocate money between Funds

View Gift

Edit permitted Gift fields

Handle concurrency

View payment status

View allocation details

✦ Engagement Management

Users need to:

EXAMPLE / FLOW

Record phone call

Record meeting

Record email

View interaction history

Create follow-up Task

Assign Task

Complete Task

See overdue Tasks

✦ Payment Management

Authorized systems/users need to:

EXAMPLE / FLOW

Create Payment

Collect through provider

Record payment method token/reference

Track attempts

Review fraud results

Receive webhooks

Retry when appropriate

Process refunds

View Payment history

✦ Settlement and Reconciliation

Finance needs to:

EXAMPLE / FLOW

View settlements

View settlement items

Compare expected payment amounts

Review fees

Review net settlement

Investigate discrepancies

Reconcile provider data

✦ Reporting

Management needs:

EXAMPLE / FLOW

```
Giving by month

Giving by year

Giving by Campaign

Giving by Fund

Top donors

Donor retention

Payment success rate

Payment failure rate

Refund totals

Processor fees

Net revenue

Fundraising interaction activity
```

✦ Administration and Operational Monitoring

Administrators need future capabilities around:

EXAMPLE / FLOW

```
failed outbox messages

failed payment events

failed webhooks

integration retries

background workers

system health

audit information
```

The exact administrative UI will be designed later.

✦ Recommended Business API Surface

These are **business capabilities**, not the final technical controller design. The .NET document will turn them into proper APIs.

Tenant capabilities

EXAMPLE / FLOW

```
Create tenant

Get tenant

Update tenant

Manage tenant users
```

Potential route family:

EXAMPLE / FLOW

```
/api/tenants
```

✦ Constituent capabilities

EXAMPLE / FLOW

```
/api/constituents

/api/constituents/{id}

/api/constituents/{id}/addresses

/api/constituents/{id}/gifts

/api/constituents/{id}/interactions

/api/constituents/{id}/tasks

/api/constituents/{id}/timeline

/api/constituents/{id}/summary
```

✦ Campaign capabilities

EXAMPLE / FLOW

```
/api/campaigns

/api/campaigns/{id}

/api/campaigns/{id}/gifts

/api/campaigns/{id}/performance

/api/campaigns/{id}/history
```

✦ Fund capabilities

EXAMPLE / FLOW

```
/api/funds

/api/funds/{id}

/api/funds/{id}/allocations

/api/funds/{id}/summary
```

✦ Gift capabilities

EXAMPLE / FLOW

```
/api/gifts

/api/gifts/{id}

/api/gifts/{id}/allocations

/api/gifts/{id}/payment
```

✦ Interaction and Task capabilities

EXAMPLE / FLOW

```
/api/interactions

/api/tasks

/api/tasks/open

/api/tasks/overdue

/api/tasks/{id}/complete
```

✦ Payment capabilities

Potential Payment API:

EXAMPLE / FLOW

```
/api/payments

/api/payments/{id}

/api/payments/{id}/attempts

/api/payments/{id}/refunds
```

✦ Webhook capability

EXAMPLE / FLOW

```
/api/payment-provider/webhooks
```

This route will be protected and validated differently from normal human-user routes.

✦ Settlement capabilities

EXAMPLE / FLOW

```
/api/settlements

/api/settlements/{id}

/api/settlements/{id}/items
```

✦ Reporting capabilities

Examples:

EXAMPLE / FLOW

```
/api/reports/giving/monthly

/api/reports/giving/yearly

/api/reports/campaigns

/api/reports/funds

/api/reports/donors/top

/api/reports/payments/success-rate

/api/reports/payments/refunds

/api/reports/payments/fees

/api/reports/interactions
```

The .NET document will decide:

EXAMPLE / FLOW

```
controllers

routes

request DTOs

response DTOs

pagination

validation

HTTP status codes

OpenAPI/Swagger

versioning

authorization policies
```

✦ Recommended Angular Business Areas

The future Angular application can be divided by actual business workflow.

Main Navigation

EXAMPLE / FLOW

```
Dashboard

Constituents

Campaigns

Funds

Gifts

Tasks

Payments

Settlements

Reports

Administration
```

✦ Dashboard

The main dashboard should eventually answer:

EXAMPLE / FLOW

```
How much did we raise?

What changed recently?

Which campaigns need attention?

How many payments failed?

How many tasks are overdue?

What requires action today?
```

Potential cards:

EXAMPLE / FLOW

```
Giving This Month

Giving This Year

Active Campaigns

Open Tasks

Overdue Tasks

Payment Success %

Failed Payments

Pending Refunds
```

✦ Constituent Workspace

A constituent profile could contain tabs:

EXAMPLE / FLOW

```
Overview

Addresses

Gifts

Interactions

Tasks

Timeline

Analytics
```

Example:

EXAMPLE / FLOW

```
John Smith
---------------------------------

Lifetime Giving
$15,500

Last Gift
$1,000

Campaigns Supported
4

Open Tasks
2

Last Interaction
Phone Call — 3 days ago
```

✦ Gift Entry Workspace

Employee can:

EXAMPLE / FLOW

```
Select constituent

Select campaign

Enter amount

Choose currency

Create allocations

Review total

Submit Gift

Start payment if appropriate
```

Allocation UI:

EXAMPLE / FLOW

```
Gift Total
$1,000

Cancer Research
$700

Patient Assistance
$300

--------------------
Allocated
$1,000

Remaining
$0
```

✦ Payment Operations Workspace

Authorized users can see:

EXAMPLE / FLOW

```
Payment status

Attempts

Failure reason

Payment method summary

Fraud result

Refunds

Settlement state
```

Never expose sensitive raw card data.

✦ Reporting Workspace

Reports could include:

EXAMPLE / FLOW

```
Executive Dashboard

Campaign Performance

Fund Performance

Donor Giving

Payment Performance

Refund Analysis

Settlement Analysis

Fundraiser Engagement
```

The Angular document will later cover:

EXAMPLE / FLOW

```
charts

tables

filters

pagination

lazy loading

state

route guards

MSAL

role-based menus

forms

validation

interceptors

error handling
```

✦ Recommended Application Roles

The exact authentication/authorization implementation belongs in the .NET and Azure documents.

At the business level, possible roles include:

EXAMPLE / FLOW

```
Platform Administrator

Tenant Administrator

Fundraising Manager

Fundraiser

Finance User

Payment Operations User

Reporting User

Read-Only User
```

✦ Platform Administrator

May manage:

EXAMPLE / FLOW

```
tenants

system administration

platform-level configuration
```

Should not automatically mean every administrative operation is done through the normal application API.

✦ Tenant Administrator

May manage:

EXAMPLE / FLOW

```
organization users

business configuration

role assignments
```

within one Tenant.

✦ Fundraising Manager

May:

EXAMPLE / FLOW

manage campaigns

manage funds

view donor information

review fundraiser work

view fundraising reports

✦ Fundraiser

May:

EXAMPLE / FLOW

manage assigned constituents

record Gifts

record interactions

create Tasks

view campaign/fund information

✦ Finance User

May:

EXAMPLE / FLOW

review Gifts

review successful Payments

manage/refine reconciliation

review settlements

review refunds

✦ Payment Operations User

May:

EXAMPLE / FLOW

```
investigate failed payment attempts

review webhooks

review provider references

process supported refund operations
```

✦ Reporting User

May:

EXAMPLE / FLOW

```
view dashboards

run reports

export appropriate data
```

without normal write access to operational business data.

✦ Data Privacy Business Rules

The platform contains sensitive data.

Examples:

EXAMPLE / FLOW

```
donor names

addresses

email

phone

gift history

payment information
```

Therefore business access should follow:

EXAMPLE / FLOW

```
Need-to-know access

Tenant isolation

Role-based access

Auditability

Secure payment handling

Encryption
```

The implementation will later use technologies such as:

EXAMPLE / FLOW

Microsoft Entra ID

OAuth 2.0 / OpenID Connect

MSAL

SQL roles/permissions

Row-Level Security where appropriate

Dynamic Data Masking

TDE

TLS

Azure Key Vault

But these belong in later technical documents.

✦ OAuth, SAML, Authentication and Authorization — Where They Fit

You specifically want to learn these topics.

They should **not** be mixed into the business model itself.

Their place will be:

EXAMPLE / FLOW

```
DOCUMENT 3
.NET / API / Security Architecture
```

We will cover there:

EXAMPLE / FLOW

```
Authentication

Authorization

OAuth 2.0

OpenID Connect

SAML concepts

Microsoft Entra ID

App registrations

Access tokens

ID tokens

Scopes

Roles

Claims

Policies

JWT Bearer authentication

MSAL

API-to-API authentication

service-to-service identity

Managed Identity

Role-based authorization
```

Business meaning first:

EXAMPLE / FLOW

```
Authentication
→ Who is this user/system?

Authorization
→ What business operation
  can this identity perform?
```

✦ Where Docker, Kubernetes, Load Balancing and GitHub Fit

They are not business tables.

They belong in:

EXAMPLE / FLOW

```
DOCUMENT 4
Azure + DevOps + Deployment
```

We will connect them to this project:

EXAMPLE / FLOW

```
GitHub
     |
     v
GitHub Actions
     |
     v
Build/Test
     |
     v
Docker Images
     |
     v
Azure Container Registry
     |
     v
App Service / AKS
     |
     v
Load Balancer / Ingress
     |
     v
.NET APIs
```

We will later cover:

EXAMPLE / FLOW

```
Git fundamentals

branches

pull requests

CI/CD

GitHub Actions

Docker

Docker Compose

container images
```

```
Kubernetes
pods
deployments
services
Ingress
load balancing
autoscaling
Azure App Service
Azure Container Apps
AKS
Azure Functions
Service Bus
Key Vault
Application Insights
OpenTelemetry
Azure SQL
Blob Storage
API Management
Managed Identity
deployment slots
environment configuration
secrets
monitoring
```

All connected to this fundraising project.

✦ Where Angular Fits

Angular belongs in:

EXAMPLE / FLOW

```
DOCUMENT 5
Angular Application Architecture
```

It will consume the APIs produced in Document 3.

We will cover:

EXAMPLE / FLOW Angular architecture

```
feature folders

routing

lazy loading

standalone components

services

HttpClient

interceptors

forms

validation

pagination

tables

filters

sorting

dashboards

charts

error handling

loading states

route guards

role-based UI

MSAL authentication

token handling
```

```
admin area

reports

payment pages

fundraising screens
```

All based on these three databases and business workflows.

✦ Where AI, Agents and MCP Fit

AI should not be added randomly.

It should help real business workflows.

This belongs in:

EXAMPLE / FLOW

```
DOCUMENT 6
AI + Agents + MCP + Automation
```

Examples for this project:

EXAMPLE / FLOW

```
Fundraising Assistant

Reporting Assistant

Donor Research Assistant

Campaign Analysis Agent

Payment Operations Assistant

Self-Service Analytics Agent

Administrative Copilot
```

✦ Example AI Use — Fundraising Copilot

A fundraiser asks:

> Give me a summary of John Smith's relationship with our organization.

The AI may combine authorized information:

EXAMPLE / FLOW

```
Constituent

Gift history

Interactions

Tasks

Campaign participation
```

and produce:

EXAMPLE / FLOW

```
John Smith has supported the organization
since 2022.

Lifetime giving: $15,500.

Most recent gift: $1,000 to the
Cancer Research Campaign.

Last interaction: phone call five days ago.

Two follow-up tasks remain open.
```

The AI must respect the same:

EXAMPLE / FLOW

```
tenant security

user authorization

data privacy
```

as the normal application.

✦ Example AI Use — Campaign Analysis Agent

Manager asks:

> Why is the Cancer Research Campaign behind target?

Agent can analyze ReportingDb:

EXAMPLE / FLOW

```
monthly giving

donor count

average gift

previous-year performance

interaction activity
```

and summarize relevant patterns.

✦ Example AI Use — Payment Operations Agent

Finance asks:

> Show me unusual payment failures today.

Agent could retrieve authorized data about:

EXAMPLE / FLOW

```
failed attempts

failure codes

fraud assessments

webhook problems
```

and produce an investigation summary.

The AI should **not** independently move money or issue refunds without an appropriately authorized workflow.

✦ MCP in This Project

MCP can provide controlled tools to AI agents.

Conceptually:

EXAMPLE / FLOW

```
AI Agent
   |
   v
MCP Server
   |
   +---- Search constituents
   |
   +---- Get campaign performance
   |
   +---- Get donor summary
   |
   +---- Get payment status
   |
   +---- Create approved Task
```

Rather than giving an AI unrestricted direct database access, MCP tools can expose specific business capabilities.

That becomes an important architecture lesson.

✦ Agentic Automation

Later you can build workflows such as:

EXAMPLE / FLOW

```
GiftCreated
     |
     v
Agent / Automation
     |
     +---- generate thank-you draft
     |
     +---- categorize donor engagement
     |
     +---- create follow-up recommendation
     |
     +---- alert fundraiser for large gift
```

or:

EXAMPLE / FLOW

```
PaymentFailed
     |
     v
Automation
     |
     +---- inspect failure
     |
     +---- determine retry eligibility
     |
     +---- notify operations team
```

Human approval should remain part of workflows where financial, privacy, or important business decisions require it.

✦ GitHub Copilot in the Learning Project

You mentioned that you already have Copilot.

Use it as a development assistant throughout the project, but not as a substitute for understanding.

Good workflow:

EXAMPLE / FLOW

```
You understand requirement
        |
        v
You design solution
        |
        v
Copilot helps generate code
        |
        v
You review code
        |
        v
Tests
        |
        v
Security review
        |
        v
Commit
```

Use Copilot for:

EXAMPLE / FLOW

```
boilerplate

unit-test suggestions

DTO mappings

SQL test-data generation

Angular component scaffolding

documentation

refactoring ideas

Dockerfile assistance

CI/CD YAML assistance
```

But verify:

EXAMPLE / FLOW

```
security

transaction correctness

SQL queries

authorization

payment logic

AI-generated migrations

production configuration
```

yourself.

The AI document will cover this deeply.

✦ Business Scenarios We Will Implement Across the Whole Learning Path

These scenarios will be reused in every technical document.

Scenario — Create Donor

EXAMPLE / FLOW

```
Employee
   |
   v
Create Constituent
   |
   v
Add Address
   |
   v
Create Follow-Up Task
```

Later teaches:

EXAMPLE / FLOW

```
SQL transaction

.NET API

Angular form

authorization

validation
```

✦ Scenario — Record Gift

EXAMPLE / FLOW

```
John Smith
    |
    v
Gift $1,000
    |
    v
Campaign
    |
    +---- Allocation $700
    |
    +---- Allocation $300
    |
    v
Outbox GiftCreated
```

Later teaches:

EXAMPLE / FLOW

```
SQL transactions

stored procedures

concurrency

.NET transaction handling

event publishing

Service Bus

Angular Gift form
```

✦ Scenario — Process Payment

EXAMPLE / FLOW

```
GiftCreated
     |
     v
Payment
     |
     v
Secure Checkout
     |
     v
PaymentAttempt
     |
     v
FraudAssessment
     |
     v
Success
     |
     v
Webhook
```

Later teaches:

EXAMPLE / FLOW

```
microservices

HTTP integrations

idempotency

webhooks

Azure Key Vault

Service Bus

retry policies
```

✦ Scenario — Refund

EXAMPLE / FLOW

```
Finance user
     |
     v
Request Refund
     |
     v
Authorization Check
     |
     v
Payment Provider
     |
     v
Refund
     |
     v
Webhook Confirmation
```

✦ Scenario — Settlement

EXAMPLE / FLOW

```
Payment provider
      |
      v
Settlement
      |
      v
Settlement Items
      |
      v
Reconciliation
      |
      v
Reporting
```

✦ Scenario — Donor Timeline

Combine:

EXAMPLE / FLOW

```
Gifts

Interactions

Tasks
```

into:

EXAMPLE / FLOW

```
Jan 10
Phone Call

Jan 15
Gift $1,000

Jan 16
Thank-You Task

Mar 01
Meeting

Mar 10
Gift $500
```

This becomes a useful SQL, API, Angular, and AI exercise.

✦ Scenario — Campaign Dashboard

Display:

EXAMPLE / FLOW

```
Campaign
Cancer Research 2026

Goal
$500,000

Raised
$375,000

Remaining
$125,000

Progress
75%

Donors
680

Gift Count
1,120
```

This will eventually exercise:

EXAMPLE / FLOW

```
SQL aggregation

ReportingDb

.NET reporting API

Angular charts

AI analysis
```

✦ Scenario — Failed Payment Operations

Dashboard:

EXAMPLE / FLOW

```
Payment #90001
Failed
Insufficient funds

Payment #90002
Webhook processing failed

Payment #90003
Fraud review required
```

Operations staff can investigate.

✦ Scenario — Executive Reporting

Leadership asks:

EXAMPLE / FLOW

```
Total giving this year?

Top five Campaigns?

Top five Funds?

Donor retention?

Payment success?

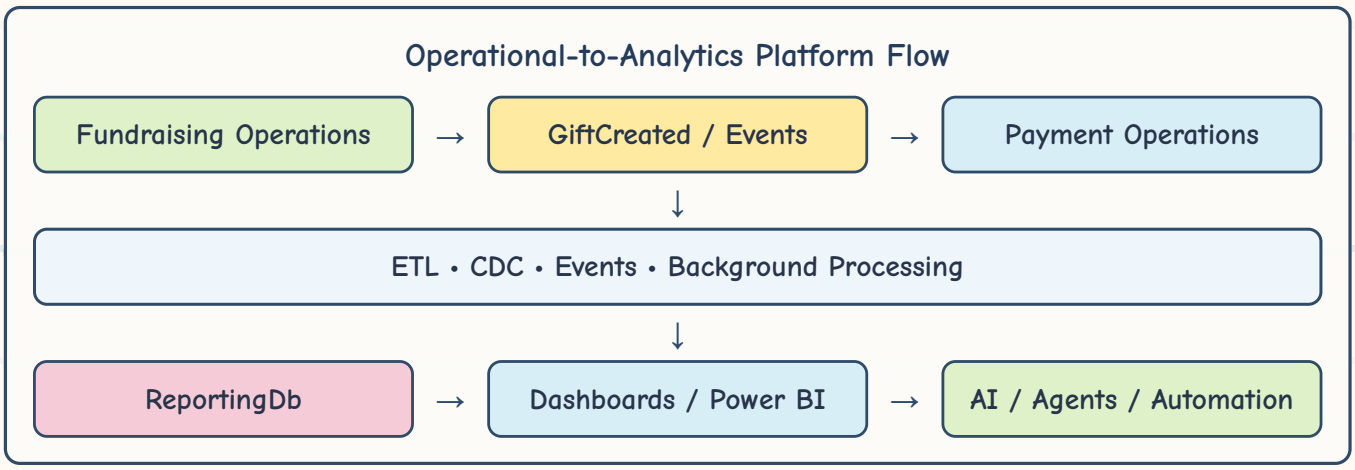
Total fees?

Total refunds?

Net collected amount?
```

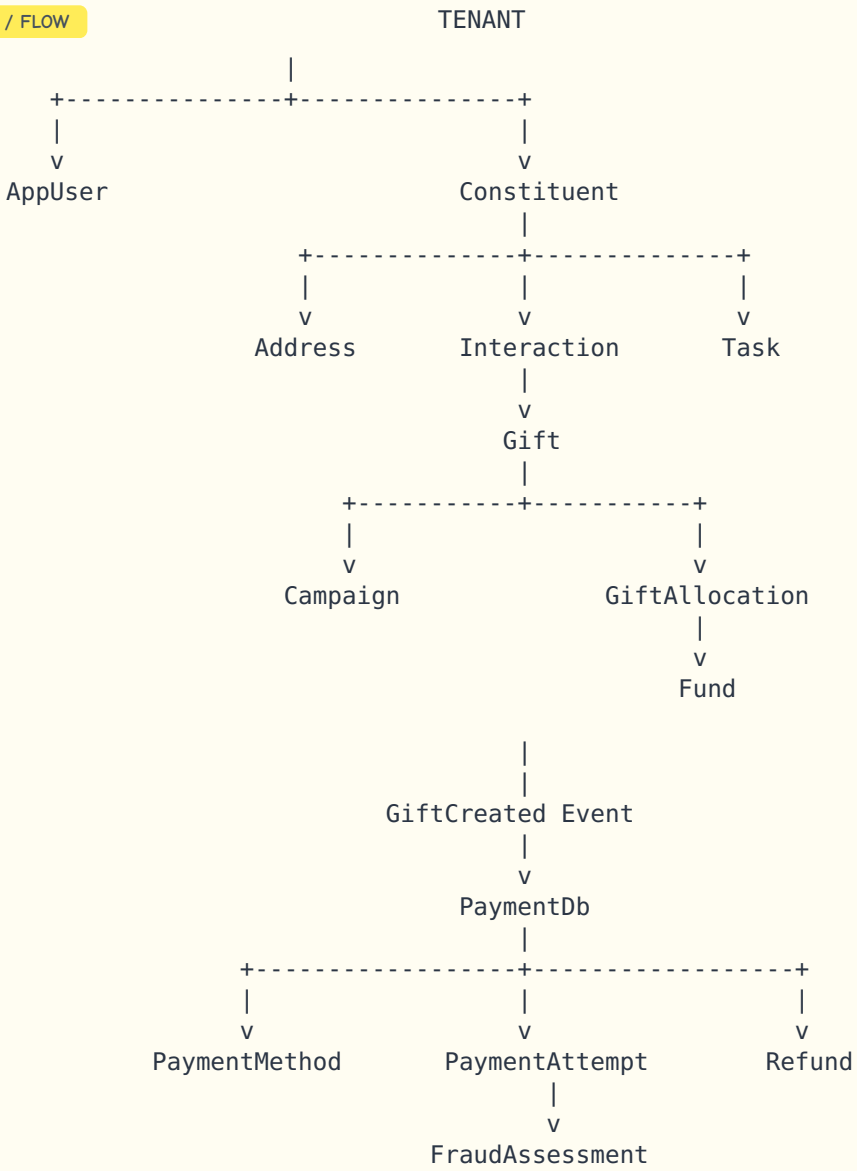
ReportingDb answers these questions.

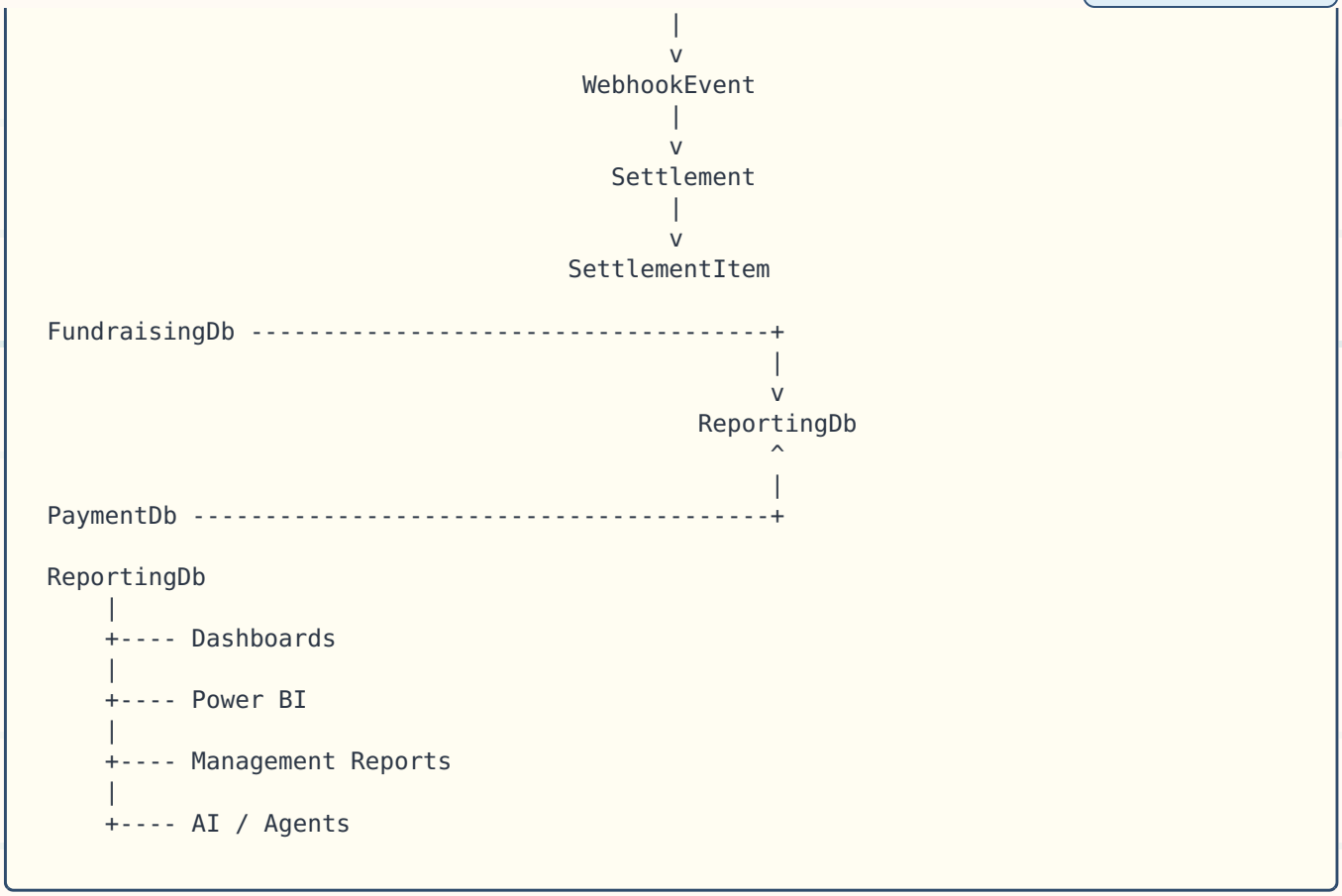
✦ Overall Platform Business Flow



The complete platform can now be understood as:

EXAMPLE / FLOW




WebhookEvent
Settlement
SettlementItem
FundraisingDb
ReportingDb
PaymentDb
ReportingDb
Dashboards
Power BI
Management Reports
AI / Agents

✦ What This Project Will Teach You

This one platform can take you from database beginner to full-stack/cloud/AI engineering.

The learning path becomes:

EXAMPLE / FLOW

```
BUSINESS DOMAIN
      |
      v
SQL SERVER
      |
      v
ASP.NET CORE / .NET
      |
      v
AUTH / SECURITY
      |
      v
EVENTS / MICROSERVICES
      |
      v
AZURE / DEVOPS
      |
      v
ANGULAR
      |
      v
AI / AGENTS / MCP
```

✦ The Document Series We Will Build

To prevent the notes from becoming shallow again, every major technical area gets its own full document.

Document 1 — Business & Domain

This document.

Covers:

EXAMPLE / FLOW

```
Business purpose

Three databases

Every major business entity

Relationships

Actors

Processes

Gift lifecycle

Payment lifecycle

Refunds

Settlements

Reporting

Business boundaries

Business capabilities

End-to-end scenarios
```

✦ Document 2 — Complete SQL Server Implementation

Next.

It will cover the three databases **end to end**, including:

EXAMPLE / FLOW

```
SQL Server project structure

CREATE DATABASE

Schemas

Every table
```

Every column

Every data type

PK

FK

UNIQUE

CHECK

DEFAULT

IDENTITY

Sequences where useful

Relationships

Normalization

OLTP design

OLAP star schema

Indexes

Clustered indexes

Nonclustered indexes

Composite indexes

Unique indexes

Filtered indexes

Covering indexes

INCLUDE

Fill factor

Fragmentation

Statistics

Views

Functions

Stored procedures

CRUD

Filtering

Sorting

Joins

Subqueries

CTEs

Recursive CTEs

Set operators

Window functions

Temp tables

Table variables

Dynamic SQL

Pagination

Transactions

TRY/CATCH

THROW

XACT_STATE

XACT_ABORT

Concurrency

Shared locks

Exclusive locks

Update locks

Intent locks

Blocking

Deadlocks

Isolation levels

RCSI

SNAPSHOT

rowversion

JSON

XML

Temporal tables

CDC

Change Tracking

Outbox

ETL concepts

Query plans

Estimated plans

Actual plans

Optimizer

Cardinality estimation

Seek

Scan

Key Lookup

Sort

Hash Match

Nested Loops

Merge Join

SARGability

Parameter sniffing

Query Store

STATISTICS IO

STATISTICS TIME

Security

Login/User

Roles

Permissions

RLS

DDM

TDE

Always Encrypted

TLS concepts

Backups

```
Recovery models

Point-in-time recovery

CHECKDB

Maintenance

Availability Groups

Failover

Replication concepts
```

And most importantly:

> **Every topic will show where it belongs in FundraisingDb, PaymentDb, or ReportingDb and which future endpoint uses it.**

✦ Document 3 — Complete .NET / ASP.NET Core Architecture

Will cover:

```
EXAMPLE / FLOW .NET solution

Clean Architecture

Domain layer

Application layer

Infrastructure layer

API layer

Dependency Injection

Controllers

Minimal APIs where appropriate

DTOs

Commands

Queries

CQRS where useful

EF Core

Dapper
```

DbContexts

Migrations

Stored procedure calls

Transactions

Unit of Work concepts

Validation

FluentValidation concepts

Exception handling

ProblemDetails

Middleware

Logging

Serilog concepts

OpenTelemetry

Pagination

Filtering

Sorting

API versioning

Swagger/OpenAPI

Health checks

Caching

Redis concepts

BackgroundService

HostedService

Outbox publisher

HttpClient

Polly/resilience

Idempotency

Webhooks

Authentication

Authorization

```
OAuth 2.0

OpenID Connect

SAML concepts

Microsoft Entra ID

JWT

Claims

Roles

Policies

Scopes

Managed Identity

MSAL relationships

API security

CORS

Rate limiting

Unit testing

Integration testing

Architecture testing
```

All using this same project.

✦ Document 4 — Complete Azure / DevOps / Containers / Microservices

Will cover:

```
EXAMPLE / FLOW Git

GitHub

branching

pull requests

GitHub Actions

CI

CD
```

Docker

Docker Compose

container registry

Azure Container Registry

Azure App Service

Azure Functions

Azure Container Apps

AKS

Kubernetes

Pods

Deployments

Services

Ingress

Load Balancing

Autoscaling

Azure SQL

Service Bus

Topics

Queues

Dead-letter queue

Azure Key Vault

Managed Identity

Azure Blob Storage

Application Insights

OpenTelemetry

API Management

Front Door concepts

Load Balancer concepts

DNS

TLS certificates

Private endpoints

VNets concepts

environment configuration

deployment slots

Infrastructure as Code concepts

Bicep/Terraform concepts

monitoring

alerts

disaster recovery

microservice communication

eventual consistency

Outbox

Saga concepts

retry

circuit breaker

distributed tracing

✦ Document 5 — Complete Angular Application

Will cover:

EXAMPLE / FLOW Angular project structure

Standalone components

Routing

Lazy loading

Feature areas

Constituent UI

Gift UI

Campaign UI

Payment UI

```
Reporting UI

Admin UI

Reactive forms

Validation

HttpClient

API services

Interceptors

Error handling

Loading states

Pagination

Filtering

Sorting

tables

charts

dashboards

state management concepts

signals

RxJS

authentication

MSAL

Entra ID

route guards

role-based navigation

token handling

responsive UI

accessibility

testing

environment configuration

production build
```

✦ Document 6 — Complete AI / Agent / MCP / Automation Layer

Will cover:

EXAMPLE / FLOW LLM fundamentals

tokens

context

prompting

structured output

tool calling

RAG

embeddings

vector search

Azure AI Search concepts

grounding

MCP

MCP Server

MCP Client

MCP tools

resources

prompts

authentication

agent architecture

single agent

multi-agent concepts

planner/executor concepts

agent memory

workflows

human approval

self-service analytics

reporting agent

```
fundraising copilot

payment operations agent

campaign agent

automation

Azure Functions + AI

Service Bus + AI

scheduled agents

event-driven agents

Copilot usage

GitHub Copilot

Copilot-assisted coding

AI security

PII protection

prompt injection

tool permission boundaries

evaluation

observability

cost management

responsible AI
```

And every AI topic will be attached to a genuine feature in this platform rather than being a separate toy project.

✦ Final Business Memory Model

When you forget the whole architecture, remember only this:

EXAMPLE / FLOW

```
FundraisingDb
=
WHO / WHY / WHERE / GIFT / RELATIONSHIP
```

EXAMPLE / FLOW

```
PaymentDb
=
HOW THE MONEY IS COLLECTED,
REFUNDED, AND SETTLED
```

EXAMPLE / FLOW

```
ReportingDb
=
WHAT THE BUSINESS CAN LEARN
FROM ALL OF THAT DATA
```

The project starts with:

EXAMPLE / FLOW

```
Constituent
```

moves through:

EXAMPLE / FLOW

```
Gift
```

then:

EXAMPLE / FLOW

```
Payment
```

then:

EXAMPLE / FLOW

```
Settlement
```

and finally:

EXAMPLE / FLOW

```
Analytics / Reporting / AI
```

The full business journey is:

EXAMPLE / FLOW

```
PROSPECT
   |
   v
CONSTITUENT
   |
   v
ENGAGEMENT
   |
   v
CAMPAIGN
   |
   v
GIFT
   |
   v
GIFT ALLOCATION
   |
   v
PAYMENT
   |
   v
PAYMENT ATTEMPT
   |
   v
SUCCESS / FAILURE
   |
   +----> REFUND
   |
   v
SETTLEMENT
   |
   v
REPORTING
   |
   v
DASHBOARDS
   |
   v
AI / AGENTS / AUTOMATION
```

That is the business foundation on which all of the following technical learning will be built.